Selezione, salti e typedef

Marco Cerbella

Lezione 14 – 26 ottobre 2026

1 Istruzioni di selezione

Un'istruzione di selezione può dirigere il flusso di esecuzione del programma lungo percorsi diversi a seconda di una condizione data. In C ci sono due istruzioni di selezione:

1. `if`;
2. `switch`.

1.1 Istruzione if

Un'istruzione `if` ha la forma

```
if ( espressione ) istruzione1 [ else istruzione2 ]
```

La clausola `else` è facoltativa. L'espressione viene valutata per prima, per determinare quale delle due istruzioni viene eseguita; deve avere tipo scalare. Se il suo valore è vero (cioè diverso da 0) viene eseguita `istruzione1`; altrimenti viene eseguita `istruzione2`, se presente.

```
1 if (testExpression)
2 {
3     // corpo dell'if
4 }
5 // istruzione subito dopo l'if
```

```
1 if (testExpression)
2 {
3     // corpo dell'if
4 }
5 else
6 {
7     // corpo dell'else
8 }
9 // istruzione subito dopo l'if...else
```

If annidati

Se più istruzioni `if` sono annidate, una clausola `else` appartiene sempre all'ultimo `if` (allo stesso livello di annidamento dei blocchi) che non ha ancora un `else`:

```
if (n > 0)
    if (n % 2 == 0)
        puts("n is positive and even");
    else    // alternativa all'ultimo if
        puts("n is positive and odd");
```

Una clausola `else` può essere assegnata a un `if` diverso racchiudendo in un blocco l'ultimo `if` che non deve avere un `else`:

```
if (n > 0)
{
    if (n % 2 == 0)
        puts("n is positive and even");
}
else        // alternativa al primo if
    puts("n is negative or zero");
```

Catena else if

Per scegliere una tra più di due istruzioni alternative si possono mettere a cascata degli `if` in una catena `else if`. Ogni nuovo `if` è semplicemente annidato nella clausola `else` di quello precedente:

```
double spec = 10.0, measured = 10.7, diff;
diff = measured - spec;
if (diff >= 0.0 && diff < 0.5)
    printf("Rounding down: %.2f\n", diff);
else if (diff >= 0.5 && diff < 1)
    printf("Rounding up: %.2f\n", diff);
else
    printf("Difference greater than 1!\n");
```

Appena una di queste espressioni è vera, viene eseguita l'istruzione corrispondente e le altre vengono scartate. Se nessuna delle condizioni è vera, viene eseguita la clausola `else` dell'ultimo `if`, se presente.

Osservazione. Nelle slide la seconda `printf` stampava ancora «Rounding down»: qui è stata corretta in «Rounding up», coerentemente con l'intervallo [0,5, 1).

1.2 Istruzione switch

Un'istruzione `switch` fa saltare il flusso di esecuzione a una tra più istruzioni, a seconda del valore di un'espressione intera:

```
switch ( espressione ) istruzione
```

`espressione` ha tipo intero e `istruzione` è il *corpo dello switch*, che contiene etichette `case` e al più un'etichetta `default`. L'espressione viene valutata una sola volta e confrontata con le espressioni costanti nelle etichette `case`. Se il valore coincide con una delle costanti, il flusso

del programma salta all'istruzione che segue quell'etichetta. Se nessuna costante coincide, il programma continua all'etichetta `default`, se c'è.

```
switch (menu())   // salta in base al risultato di menu()
{
    case 'A': action1();     // esegue l'azione 1
              break;         // non esegue nessun'altra "azione"
    case 'B': action2();     // esegue l'azione 2
              break;         // non esegue l'azione di default
    default:  putchar('\a'); // nessun comando riconosciuto
}
```

Intervalli

Per trattare un intervallo di valori con la stessa azione, si possono scrivere più etichette consecutive:

```
switch (value)
{
    case 1:
    case 2:
    case 3:
    case 4:
    case 5:
    case 6:
    case 7:
    case 8:
        printf("Hello, 1 to 8\n");
        break;
    default:
        printf("Hello, default\n");
        break;
}
```

Esiste anche un'estensione di GCC, non standard, che permette di scrivere `case 1 ... 8:`. Un'alternativa è mettere nel `default` un `if`:

```
switch (value) {
    case 1:
        /* do 1 */
        break;
    case 2:
        /* do 2 */
        break;
    default:
        if (value >= 3 && value <= 8)
            printf("Hello, 3 to 8\n");
}
```

Se però si deve lavorare con molti intervalli, conviene usare gli `if`.

Senza break: fall-through

Senza `break` l'esecuzione prosegue nelle etichette successive.

Esempio.

```c
#include <stdio.h>
int main()
{
    int Grade = 'A';
    switch (Grade)
    {
        case 'A': printf("Excellent\n");
        case 'B': printf("Good\n");
        case 'C': printf("OK\n");
        case 'D': printf("Mmmmm....\n");
        case 'F': printf("You must do better than this\n");
        default : printf("What is your grade anyway?\n");
    }
}
```

```
Excellent
Good
OK
Mmmmm....
You must do better than this
What is your grade anyway?
```

2 Salti incondizionati

Le istruzioni di salto interrompono l'esecuzione sequenziale delle istruzioni, in modo che l'esecuzione continui in un altro punto del programma. Un salto distrugge le variabili automatiche se la destinazione del salto è fuori dal loro scope. In C ci sono quattro istruzioni che causano salti incondizionati: `break`, `continue`, `goto` e `return`.

2.1 break

L'istruzione `break;` può comparire solo nel corpo di un ciclo o di uno `switch` e causa un salto alla prima istruzione dopo il ciclo o lo `switch` che la contiene immediatamente. Si può quindi usare `break` per terminare l'esecuzione di un ciclo in qualunque punto del corpo.

Esempio.

```c
#include <stdio.h>
int main()
{
    int i;
    double number, sum = 0.0;

    for (i = 1; i <= 10; ++i)
    {
```

```
9           printf("Enter a n%d: ", i);
10          scanf("%lf", &number);
11
12          // se l'utente inserisce un numero negativo, il ciclo termina
13          if (number < 0.0)
14          {
15              break;
16          }
17          sum += number;
18      }
19      printf("Sum = %.2lf", sum);
20      return 0;
21  }
```

Con input 2,4, 4,5, 3,4, −3 l'output è `Sum = 10.30`.

2.2 continue

L'istruzione `continue;` si può usare solo nel corpo di un ciclo e fa saltare il resto dell'iterazione corrente.

- In un ciclo `while` o `do ... while` il programma salta alla successiva valutazione dell'espressione di controllo.
- In un ciclo `for` il programma salta alla successiva valutazione della terza espressione dell'istruzione `for`, che contiene le operazioni eseguite dopo ogni iterazione.

Esempio. Stesso programma di prima, ma con `continue` al posto di `break`:

```
1  for (i = 1; i <= 10; ++i)
2  {
3      printf("Enter a n%d: ", i);
4      scanf("%lf", &number);
5
6      // se l'utente inserisce un numero negativo, si passa all'iterazione
       successiva
7      if (number < 0.0)
8      {
9          continue;
10     }
11     sum += number;
12 }
```

Con input 1,1, 2,2, 5,5, 4,4, −3,4, −45,5, 34,5, −4,2, −1000, 12 i negativi vengono saltati e l'output è `Sum = 59.70`.

2.3 goto

L'istruzione `goto` causa un salto incondizionato a un'altra istruzione nella stessa funzione. La destinazione del salto è specificata dal nome di un'*etichetta*:

```
goto nome_etichetta;
```

Un'etichetta è un nome seguito da due punti: `nome_etichetta: istruzione`. Le etichette hanno un proprio spazio dei nomi, quindi possono avere gli stessi nomi di variabili o tipi senza

causare conflitti. Le etichette possono essere messe prima di qualunque istruzione, e un'istruzione può avere più etichette. Servono solo come destinazioni dei `goto` e non hanno alcun effetto se l'istruzione etichettata viene raggiunta nel normale corso dell'esecuzione sequenziale.

```c
void calculate(int a)
{
    if (a < 1 || a > 5)
        goto here;
    printf("a between 1 and 5!\n");
    return;
here: printf("a < 1 or a > 5!\n");
    return;
}
```

Osservazione. Poiché il codice che fa largo uso di `goto` è difficile da leggere, conviene usarli solo quando offrono un vantaggio evidente, come l'uscita rapida da cicli profondamente annidati. Qualunque programma C che usa `goto` si può scrivere anche senza.

Lo *spaghetti code* è un'espressione spregiativa per il codice sorgente con una struttura di controllo complessa e ingarbugliata, soprattutto quando usa molti `goto`, eccezioni, thread o altri costrutti di salto «non strutturati» (tipico di assembly, Fortran, BASIC).

2.4 return

Si veda la lezione sulle funzioni: `return` è anch'esso un'istruzione di salto incondizionato, come `goto`.

3 typedef

Il modo più semplice di usare tipi con nomi complessi è dichiarare dei *sinonimi* semplici per essi. In una dichiarazione che inizia con la parola chiave `typedef`, ogni dichiaratore definisce un identificatore come sinonimo del tipo specificato. L'identificatore è poi detto *typedef name* per quel tipo. A parte la parola chiave `typedef`, la sintassi è esattamente la stessa di una dichiarazione di un oggetto o di una funzione del tipo specificato.

Esempio.

```c
typedef unsigned int UINT;
typedef enum state {DEAD, ALIVE} State;

UINT ui = 10;
State s = DEAD;
```

Nello scope di queste dichiarazioni `UINT` è sinonimo di `unsigned int` e `State` è sinonimo di `enum state`. La variabile `ui` ha tipo `unsigned int` e `s` ha tipo `enum state`.

4 Riferimenti

Sul libro: sezione 10.6 (`typedef`).